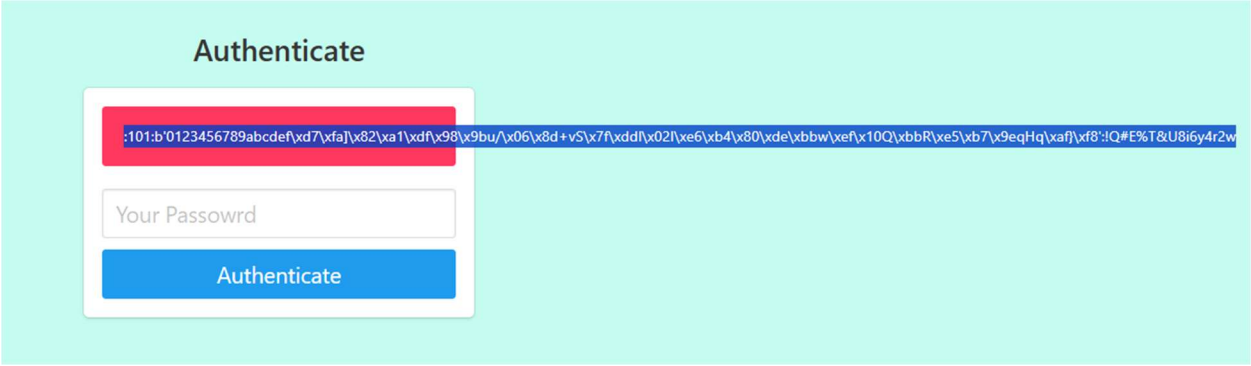
Test of Patch # 1: SQL Injection

Pre-patch:

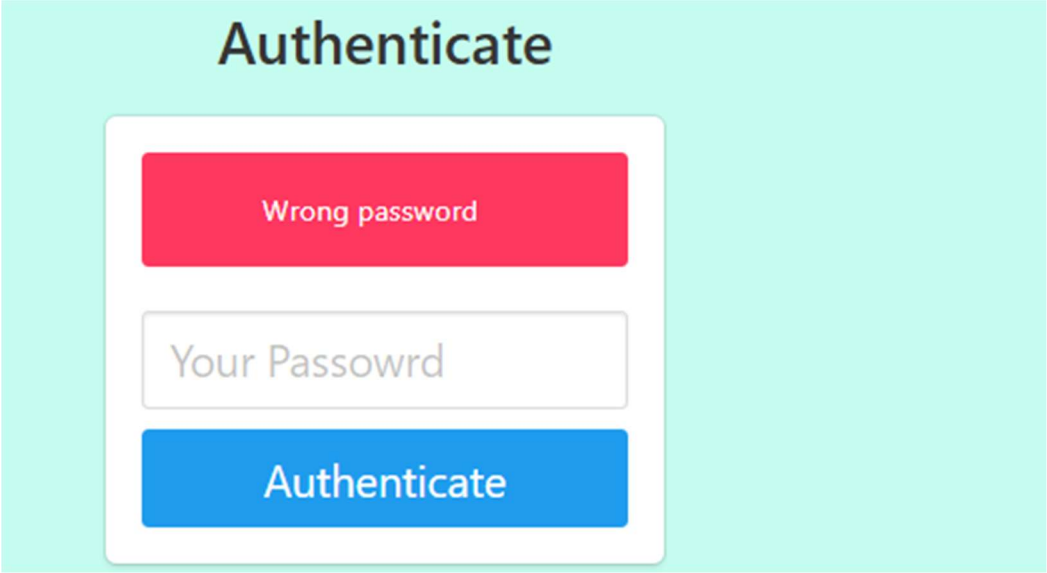
Before patching the vulnerability, when I write ' OR '1'='1 in the password field, then the code concatenates it to the SQL query. As a result, it returns “:101:b'0123456789abcdef\xd7\xfa]\x82\xa1\xdf\x98\x9bu/\x06\x8d+vS\x7f\xddl\x02l\xe6\xb4\x80\xde\xbbw\xef\x10Q\xbbR\xe5\xb7\x9eqHq\xaf}\xf8':!Q#E%T&U8i6y4r2w”

Which most probably contains the user ID, encrypted password, and the key.



After-patch:

When anything other than the correct password is entered, the error message will be “Wrong password” as shown below:



The screenshot below confirms that the application is still functional after applying the patch.

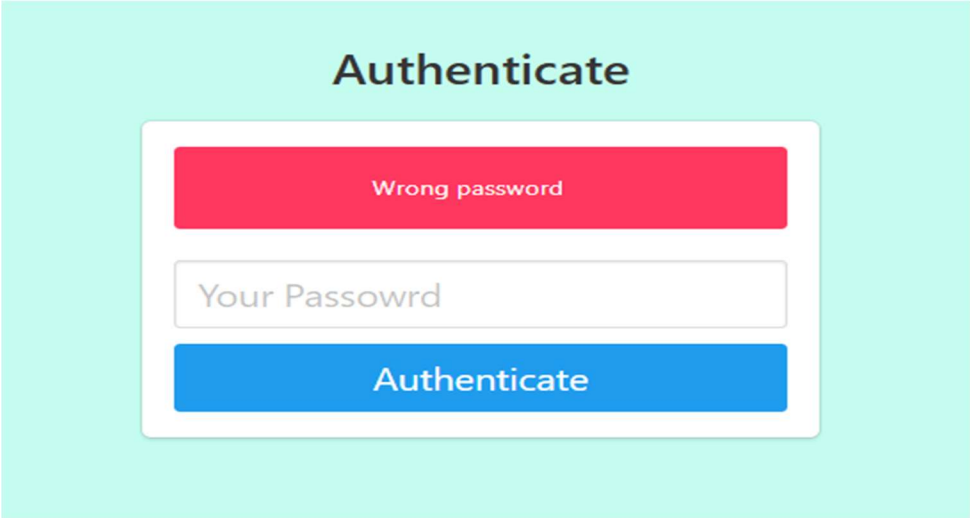
Test of Patch # 2: No Rate limiting in case of using wrong password

Pre-patch:

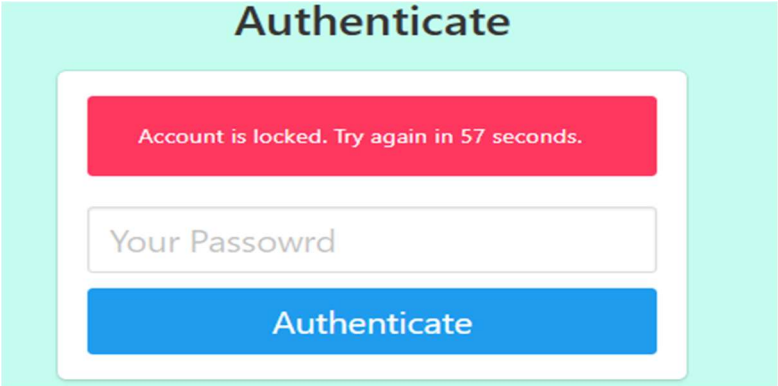
I entered wrong password many times, then I entered the correct password, and the application logged me in. This is exactly the vulnerability.

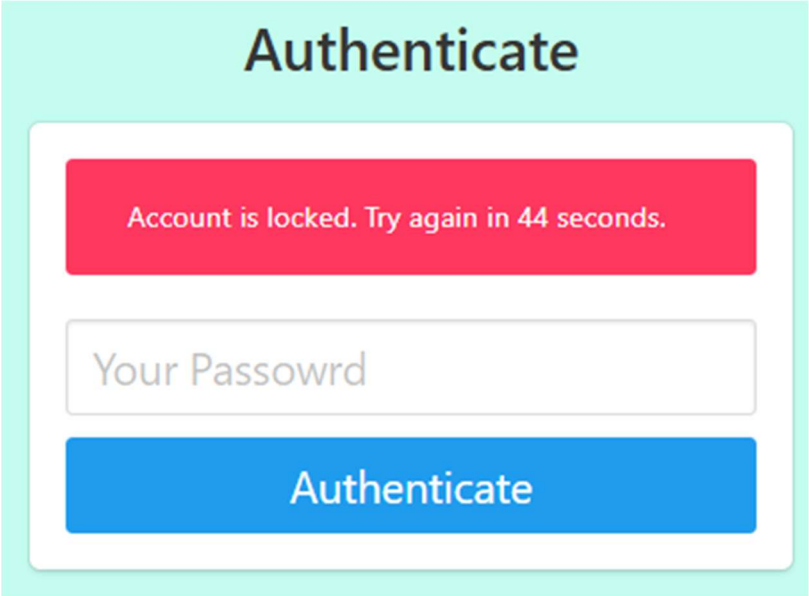
After-patch:

The first 3 failure attempts, the user will get the following message:

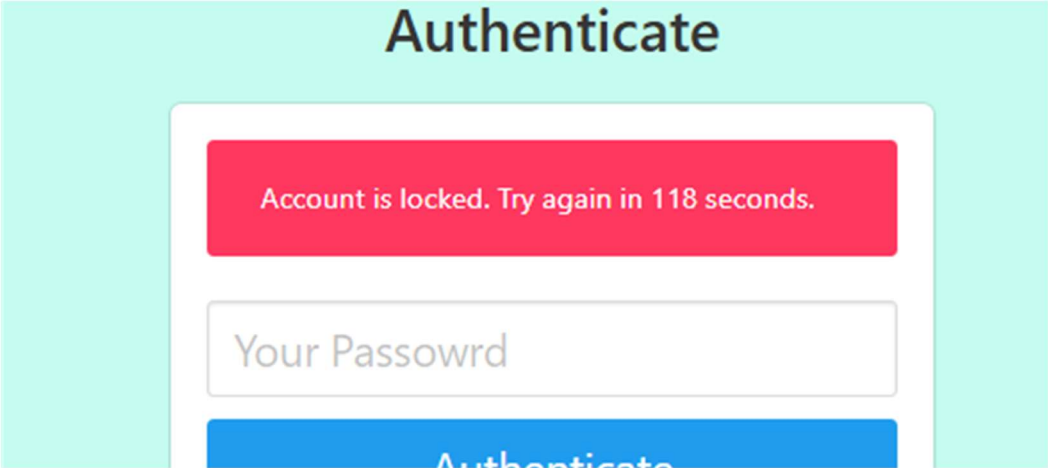


After the third failure the user will be locked and will get the following messages:





After the timer went off and the user tried wrong password again then they will be locked for 120 seconds. The locked-out period would be 180 seconds next time and so on:

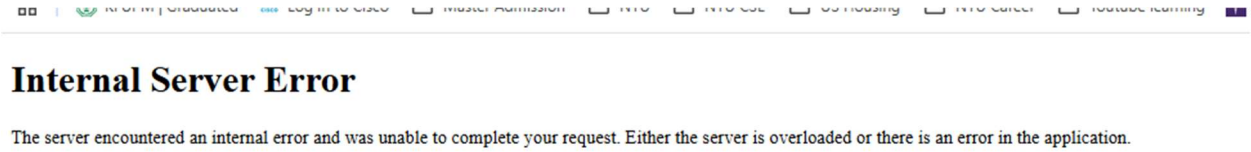


But if the user enters the correct password after the locked-out period passes, the account will go back to normal. **The screenshot below confirms that the application is still functional after applying the patch.**

Test of Patch # 3: Missing exception handling in DB connection

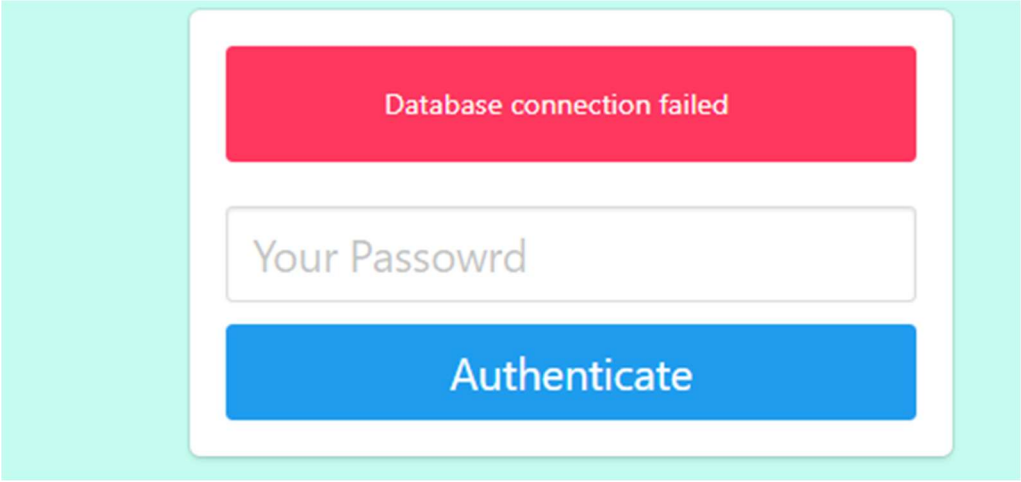
Pre-patch:

For testing the issue before patching, I removed the database from the folders to see how the code would handle this issue. As shown in the screenshot below, the web page crashed with no reasonable error that described the real issue.

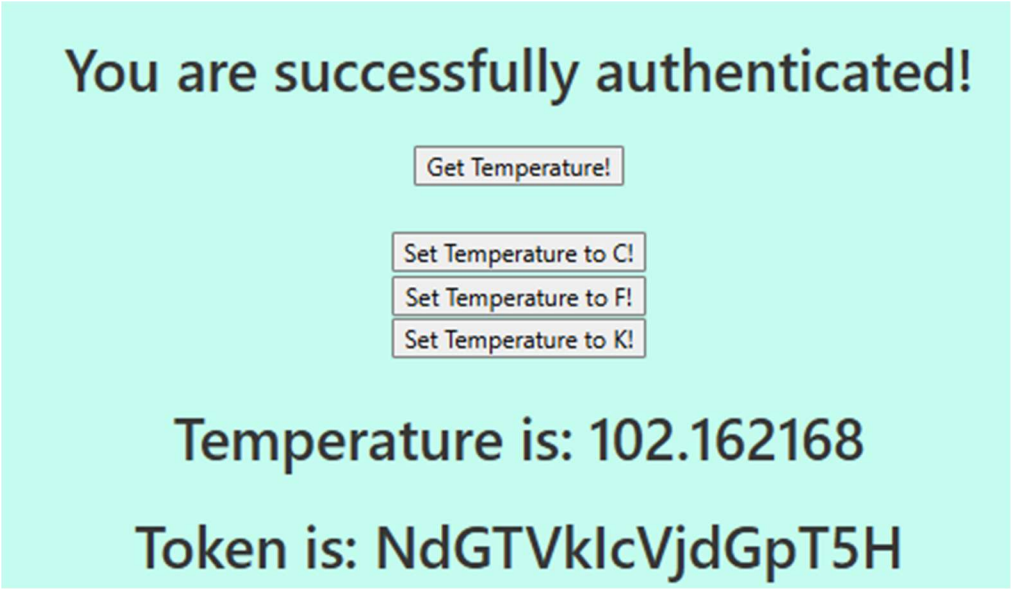


After-patch:

As the screenshot below shows that when an error is related to the database, the system will not crash. It will show the real error message.



After I moved the database back to the folder, the application works as expected as shown below:



Test of Patch # 4: Multiple command process at one authentication request

Pre-patch:

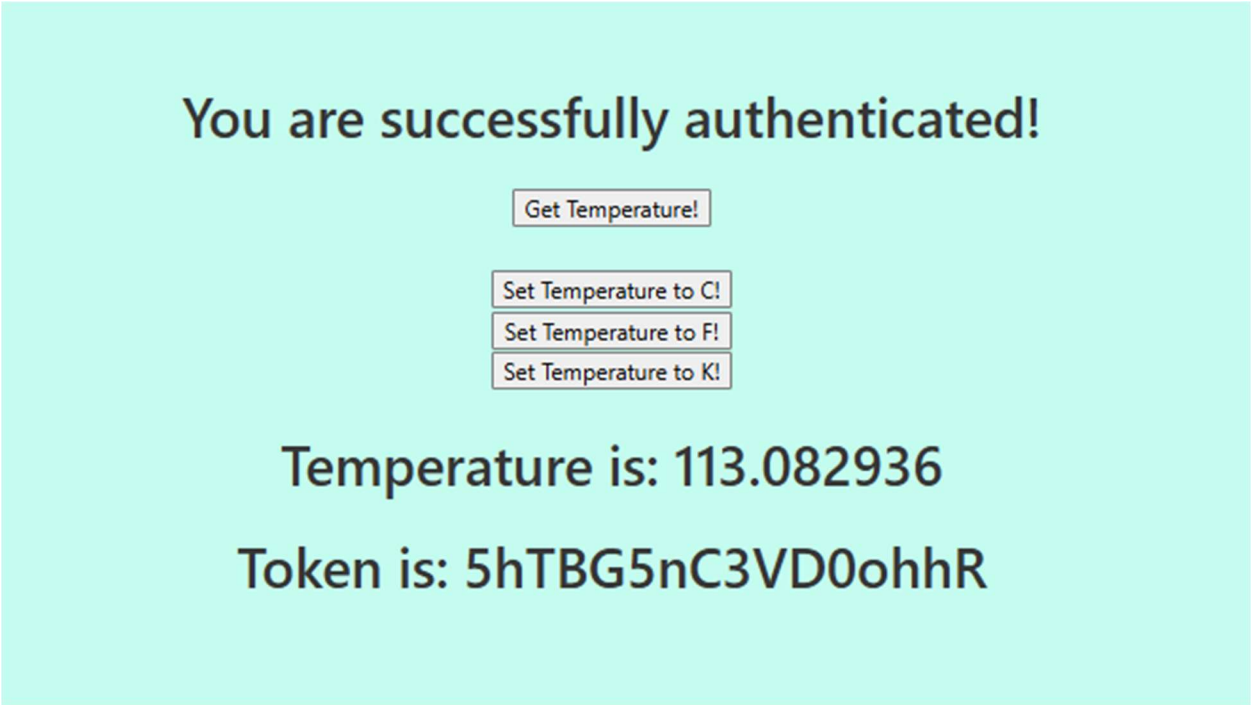
To test the vulnerability, I created a code that sends one request with multiple commands to the server and proves that the server accepted both commands. Note that the code will be included in the submission in addition to REED ME note that contains instructions on how to run the code.

```
import socket
from Crypto.Cipher import AES
import SampleNetworkClient
import os

ip = "127.0.0.1"
port = 23456

## in lab 3 I made the communication between the client and the server encrypted, so these keys
#must be loaded to the OS to communicate with the server
shared_key = os.environ.get("NETWORK_KEY").encode("utf-8")
password = os.environ.get("SERVER_PASSWORD").encode("utf-8")

def encrypt(message):
    cipher = AES.new(shared_key, AES.MODE_EAX)
    ciphertext, tag = cipher.encrypt_and_digest(message.encode("utf-8"))
    return cipher.nonce + tag + ciphertext

def decrypt(message):
    nonce = message[:16]
    tag = message[16:32]
    ciphertext = message[32:]
    cipher = AES.new(shared_key, AES.MODE_EAX, nonce=nonce)
    plaintext = cipher.decrypt_and_verify(ciphertext, tag)
    return plaintext.decode("utf-8")

# Step 1: get a valid token using the current client
snc = SampleNetworkClient.SimpleNetworkClient(23456, 23457)
token = snc.authenticate(port, password).decode("utf-8")
print("Token:", token)

# Step 2: send a multi-command payload directly

message = f"{token};SET_DEGC;GET_TEMP"

client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client_socket.settimeout(3)

payload = encrypt(message)
client_socket.sendto(payload, (ip, port))

try:
    response, _ = client_socket.recvfrom(1024)
    print("Server response:", decrypt(response))
except socket.timeout:
    print("No response from server")
except Exception as ex:
    print("Error:", ex)
```

```
(lab4env) ali@Ali:/mnt/c/Users/alsai/OneDrive/Documents/NYU/Spring 2026/Information Systems Securi
ty Engineering and Management (CS-GY 6803)/Labs/Lab 4$ python3 Vulnerability4_test.py
Token: 0yT8wUVkzCXfByp2
Server response: 45.049036
```

After-patch:

After I patch the vulnerability, I test the same code that I have developed, and I got “Invalid Command” as a response.

```
(lab4env) ali@Ali:/mnt/c/Users/alsai/OneDrive/Documents/NYU/Spring 2026/Information Systems Securi
ty Engineering and Management (CS-GY 6803)/Labs/Lab 4$ python3 Vulnerability4_test.py
Token: Y9KxZPiHFLA4sqXO
Server response: Invalid Command
```

But still the single command request is working as shown below: